

Lists II — Slicing and Built-ins

Lecture 18

CS 1001

Kirk Duran

Today

- ▶ Extract portions of a list using slicing.
- ▶ Interpret start, stop, and the optional step.
- ▶ Distinguish operations that create a new list from operations that mutate an existing list.
- ▶ Measure list length with `len()`.
- ▶ Add elements using `append()`, `extend()`, concatenation, and `insert()`.
- ▶ Remove elements using `pop()` and `remove()`.
- ▶ Refine the list memory model to account for changing list size.

Key idea

A Python list behaves like a resizable sequence: positions remain ordered while Python manages the storage required when the sequence grows or shrinks.

Part 1: Slicing a Sequence

Indexing Selects One Element; Slicing Selects a Range

Indexing

`values[2] → 30`

- ▶ Selects one position.
- ▶ Evaluates to one element value.

Slicing

`values[1:4] → [20, 30, 40]`

- ▶ Selects a sequence of positions.
- ▶ Evaluates to a new list.

Key idea

Indexing asks for one element. Slicing asks for a subsequence.

Definition: Slice

Definition

A **slice** specifies a range of positions from a sequence and produces a new sequence containing the selected elements.

General list-slice syntax

```
list[start:stop:step]
```

- ▶ **start**: first selected index.
- ▶ **stop**: boundary index at which selection stops.
- ▶ **step**: distance between selected indexes.
- ▶ Any of these components may be omitted when its default behavior is desired.

The Stop Index Is Exclusive

Example

```
values = [10, 20, 30, 40, 50]
```

```
values[1:4] → [20, 30, 40]
```

index	0	1	2	3	4
value	10	20	30	40	50

Key idea

start is included. stop is not included. The slice 1:4 selects indexes 1, 2, and 3.

Omitted Slice Bounds Use Sequence Boundaries

Examples

```
values = [10, 20, 30, 40, 50]
```

```
values[:3] → [10, 20, 30]
```

```
values[2:] → [30, 40, 50]
```

```
values[:] → [10, 20, 30, 40, 50]
```

- ▶ With a positive step, omitted start begins at the first element.
- ▶ Omitted stop continues through the final element.
- ▶ Omitting both bounds selects the complete list and produces a new list.

Slice Bounds May Extend Beyond the Valid Index Range

Examples

```
values = [10, 20, 30, 40]
```

```
values[1:100] → [20, 30, 40]
```

```
values[100:200] → []
```

- ▶ Ordinary indexing such as `values[100]` raises `IndexError`.
- ▶ Slicing adjusts an out-of-range boundary to the available sequence.
- ▶ A valid slice may therefore produce an empty list.

Negative Indexes Also Work in Slice Bounds

Example

```
values = [10, 20, 30, 40, 50]
```

```
values[1:-1] → [20, 30, 40]
```

- ▶ -1 identifies the final element's position.
- ▶ Because stop is exclusive, the element at index -1 is not selected.
- ▶ The same index rules from Lecture 17 apply inside slice notation.

Part 2: The Optional Step

The Step Controls the Distance Between Selected Indexes

Example

```
values = [10, 20, 30, 40, 50, 60]
```

```
values[0:6:2] → [10, 30, 50]
```

selected index	0	2	4
selected value	10	30	50

- ▶ If step is omitted, it defaults to 1.
- ▶ A step of 0 is invalid because the index would never advance; Python raises `ValueError`.

A Negative Step Selects Positions in Decreasing Index Order

Example

```
values = [10, 20, 30, 40, 50]
```

```
values[4:1:-1] → [50, 40, 30]
```

- ▶ Start at index 4.
- ▶ Subtract 1 after each selected position.
- ▶ Stop before index 1.

Key idea

The stop boundary remains exclusive even when the direction is reversed.

The Slice `[::-1]` Produces a Reversed Copy

Code

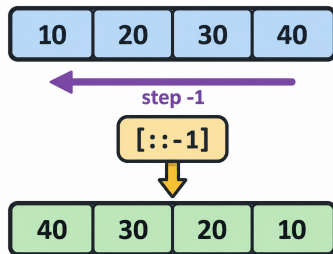
```
values = [10, 20, 30, 40]  
reversed_values = values[::-1]
```

Result

```
reversed_values → [40, 30, 20, 10]
```

Important

The original list is not mutated by the slice.



Practice: Predict the Slice Results

Given

```
data = [5, 10, 15, 20, 25, 30]
```

In class

Predict each result before evaluating it in Python.

1. `data[1:4]`
2. `data[:3]`
3. `data[3:]`
4. `data[::2]`
5. `data[4:1:-1]`
6. `data[::-1]`

Part 3: Length and Operation Categories

`len()` Returns the Number of Elements

Definition

The built-in function `len(sequence)` returns the number of elements in the sequence as an integer.

Examples

`len([10, 20, 30, 40])` $\rightarrow$ 4

`len([])` $\rightarrow$ 0

Key idea

Length counts elements. It is not an index.

Length and the Last Positive Index Differ by One

Example

```
values = [10, 20, 30, 40]
```

```
len(values) → 4
```

```
values[3] → 40
```

- ▶ Four elements occupy positive indexes 0, 1, 2, and 3.
- ▶ Therefore, the final valid positive index is `len(values) - 1`.
- ▶ `values[len(values)]` would be one position beyond the list and raises `IndexError`.

Built-in Functions and List Methods Use Different Syntax

Built-in function

`len(values)`

- ▶ The list is passed as an argument.
- ▶ The call returns a result.

List method

`values.append(40)`

- ▶ The method is invoked through the list value.
- ▶ Many list methods mutate that list.

Part 4: Growing a List

`append()` Adds One Element at the End

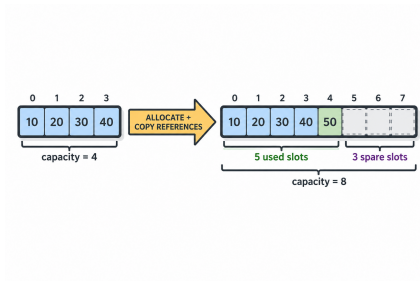
Example

```
numbers = [10, 20, 30]  
numbers.append(40)  
  
numbers → [10, 20, 30, 40]
```

- ▶ The existing list is mutated.
- ▶ Its logical length increases by one.
- ▶ The supplied argument becomes exactly one new element.
- ▶ `append()` returns `None`.

Growing a List May Require Resizing Its Storage

- ▶ In our working model, adding an element requires another adjacent slot.
- ▶ Python manages this storage change automatically.
- ▶ In CPython, the list uses a contiguous array of object references with some internal capacity.
- ▶ If growth exceeds available capacity, Python may allocate a larger reference array and copy the existing references.



Key idea

Dynamic resizing is hidden behind the list abstraction; the programmer specifies the sequence operation, not the memory-management procedure.

`extend()` Adds the Elements of Another List

Example

```
numbers = [10, 20, 30]
numbers.extend([40, 50])
numbers → [10, 20, 30, 40, 50]
```

- ▶ The original list is mutated.
- ▶ Each element from the supplied list becomes a separate element at the end.
- ▶ The logical length increases by the number of added elements.
- ▶ `extend()` returns `None`.

append() and extend() Are Not Equivalent

Append one value

```
a = [10, 20]
a.append([30, 40])

a →
[10, 20, [30, 40]]
```

Extend by values

```
b = [10, 20]
b.extend([30, 40])

b →
[10, 20, 30, 40]
```

Key idea

append(x) adds x as one element. extend(other) adds the elements contained in other.

Concatenation Combines Lists Without Mutating Either Operand

Example

```
left = [10, 20]  
right = [30, 40]  
combined = left + right
```

State afterward

```
left → [10, 20]  
right → [30, 40]  
combined → [10, 20, 30, 40]
```

Key idea

Concatenation creates a new list; `extend()` changes an existing list. Similar visible contents can arise from different state-change semantics.

`insert()` Adds One Element at a Specific Position

Example

```
numbers = [10, 20, 40, 50]
numbers.insert(2, 30)
numbers → [10, 20, 30, 40, 50]
```

- ▶ The first argument specifies the insertion position.
- ▶ The second argument is the value to insert.
- ▶ The existing element at that position and later elements move right.
- ▶ `insert()` mutates the list and returns `None`.

Part 5: Removing Elements

pop() Removes by Position and Returns the Removed Element

Example

```
numbers = [10, 20, 30, 40]  
removed = numbers.pop(1)
```

State afterward

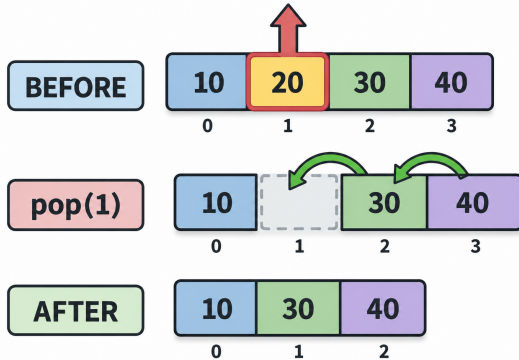
```
removed → 20  
numbers → [10, 30, 40]
```

- ▶ With no index argument, pop() removes and returns the final element.
- ▶ An out-of-range supplied index raises `IndexError`.

Key idea

pop() both mutates the list and produces the removed element as its return value.

Removing from the Middle Shifts Later Positions Left



- ▶ Removing a middle element creates a gap in the ordered positions.
- ▶ Later elements shift left so the list continues to use consecutive indexes starting at 0.
- ▶ The logical length decreases by one.

`remove()` Removes by Value

Example

```
fruits = ["apple", "banana", "cherry", "banana"]  
fruits.remove("banana")  
  
fruits → ["apple", "cherry", "banana"]
```

- ▶ `remove(value)` searches for the first equal value.
- ▶ That first matching element is removed.
- ▶ The method mutates the list and returns `None`.
- ▶ If no matching value exists, Python raises `ValueError`.

Part 6: Reversal, Resizing, and Tracing

Reversed Slice Versus `reverse()`

New reversed list

```
a = [1, 2, 3]
```

```
b = a[::-1]
```

```
a → [1, 2, 3]
```

```
b → [3, 2, 1]
```

Mutate existing list

```
a = [1, 2, 3]
```

```
a.reverse()
```

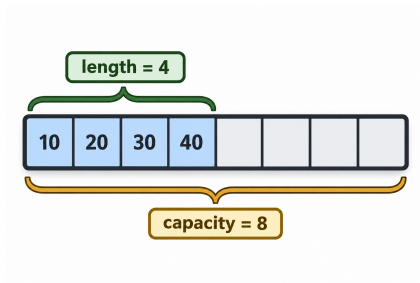
```
a → [3, 2, 1]
```

Key idea

Similar visible results can have different mutation semantics.

List Size Is Logical; Capacity Is an Implementation Detail

- ▶ `len(values)` reports the logical number of elements.
- ▶ The underlying storage may reserve additional capacity for future growth.
- ▶ Adding or removing elements changes the logical sequence immediately.
- ▶ Any necessary allocation, copying, or capacity management is performed by Python.



Trace State After Each Mutating Operation

Code

```
values = [10, 20, 30]
values.append(40)
values.insert(1, 15)
removed = values.pop(3)
```

After statement	values	removed
creation	[10, 20, 30]	—
append(40)	[10, 20, 30, 40]	—
insert(1, 15)	[10, 15, 20, 30, 40]	—
pop(3)	[10, 15, 20, 40]	30

Trace New-List Operations Separately from Mutation

Code

```
values = [10, 20, 30, 40]
middle = values[1:3]
combined = values + [50, 60]
```

Name	Value afterward
values	[10, 20, 30, 40]
middle	[20, 30]
combined	[10, 20, 30, 40, 50, 60]

Key idea

Neither slicing nor concatenation changes values in this trace.

Operation Summary

Operation	Mutates list?	Result of expression/call
<code>a[start:stop:step]</code>	no	new list
<code>len(a)</code>	no	integer length
<code>a.append(x)</code>	yes	None
<code>a.extend(b)</code>	yes	None
<code>a + b</code>	no	new list
<code>a.insert(i, x)</code>	yes	None
<code>a.pop(i)</code>	yes	removed element
<code>a.remove(x)</code>	yes	None
<code>a.reverse()</code>	yes	None

Integrated Prediction Exercise

Code

```
data = [5, 10, 15, 20]
part = data[1:3]
data.append(25)
data.insert(2, 12)
removed = data.pop()
data.reverse()
```

In class

Without running the program, determine the final values of `data`, `part`, and `removed`. For every statement, classify it as mutating or non-mutating.

Takeaways

- ▶ Slicing uses `start:stop:step`; the stop boundary is exclusive.
- ▶ Omitting slice components invokes defaults, and a negative step reverses the direction of selection.
- ▶ `len()` reports the number of elements, not the last index.
- ▶ `append()`, `extend()`, `insert()`, `pop()`, `remove()`, and `reverse()` mutate a list.
- ▶ Slicing and concatenation create new lists instead of mutating their operands.
- ▶ Python automatically manages storage as a list grows or shrinks; our programming model remains an ordered sequence of adjacent positions.

Key idea

When tracing list code, always track both the sequence contents and whether an operation mutates an existing list or produces a new value.